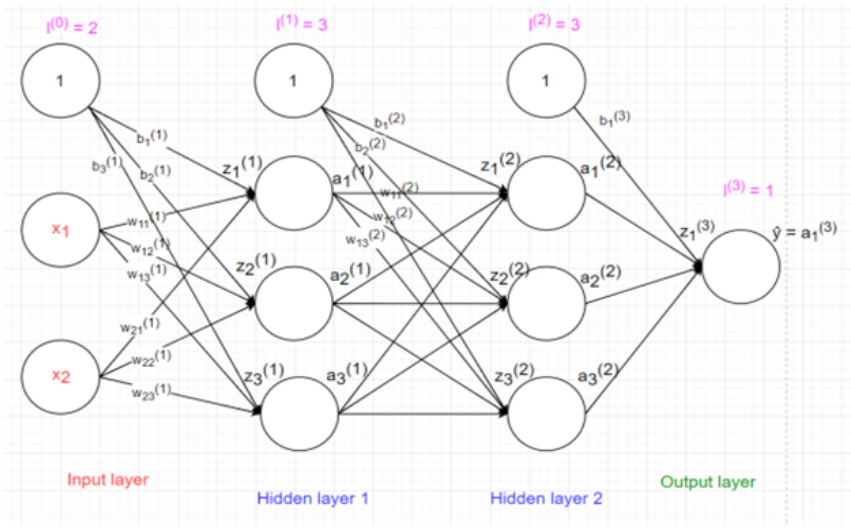
Chương 2

MẠNG LAN TRUYỀN ĐA TẦNG

Chương 2 sẽ đi sâu vào Mạng Lan Truyền Đa Tầng (Multilayer Perceptron), một kiến trúc nền tảng trong học sâu. Chương này bắt đầu bằng việc giới thiệu thuật toán lan truyền xuôi, là cách dữ liệu được xử lý đi qua mạng. Tiếp theo, trọng tâm chính là thuật toán lan truyền ngược (Backpropagation) - một cơ chế cốt lõi để huấn luyện mạng, bao gồm mục tiêu, cách thức hoạt động, cùng với quá trình cập nhật trọng số. Phần cuối của chương trình bày phương pháp luận, thảo luận về độ đo hiệu năng, cách thu thập thêm dữ liệu và tinh chỉnh các siêu tham số, cung cấp một cái nhìn toàn diện về việc xây dựng và tối ưu hóa các mạng nơ-ron đa tầng.

2.1 Thuật toán lan truyền xuôi với mạng truyền thẳng nhiều tầng



Hình 2.1: Mô hình mạng neural truyền thẳng.

Số node trong hidden layer thứ i là l^i.

Ma trận $W^{(k)}$ kích thước $l^{(k-1)} \times l^{(k)}$ là ma trận hệ số giữa layer (k-1) và layer k, trong đó $w_{i,j}^{(k)}$ là hệ số kết nối từ node thứ i của layer k-1 đến node thứ j của layer k.

Vector b^k kích thước $l^{k-1} \times 1$ là hệ số bias của các node trong layer k, trong đó b_i^k là bias của node thứ i trong layer k.

Với node thứ i trong layer l có bias $b_i^(l))$ thực hiện 2 bước:

- Tính tổng linear:

$$z_i^{(l)} = \sum_{j=1}^{l^{(l-1)}} \left(a_j^{(l-1)} w_{ji}^{(l)} \right) + b_i^{(l)}, \tag{2.1}$$

là tổng tất cả các node trong layer trước nhân với hệ số w tương ứng, rồi cộng với bias b.

- Áp dụng activation function: $a_i^{(l)} = \sigma\left(z_i^{(l)}\right)$

Vector $z^(k))$ kích thước $l^(k)) \times 1$ là giá trị các node trong layer k sau bước tính tổng linear.

Vector $a^(k))$ kích thước $l^(k)) * 1$ là giá trị của các node trong layer k sau khi áp dụng hàm activation function.

Mô hình neural network trên gồm 3 layer. Input layer có 2 node $l^(0)) = 2$, hidden layer 1 có 3 node, hidden layer 2 có 3 node và output layer có 1 node.

Do mỗi node trong hidden layer và output layer đều có bias nên trong input layer và hidden layer cần thêm node 1 để tính bias (nhưng không tính vào tổng số node layer có).

Tại node thứ 2 ở layer 1, ta có:

- $z_2^{(1)} = x_1 w_{12}^{(1)} + x_2 w_{22}^{(1)} + b_2^{(1)}$
- $a_2^{(1)} = \sigma\left(z_2^{(1)}\right)$

Hay ở node thứ 3 layer 2, ta có:

- $z_3^{(2)} = a_1 w_{13}^{(2)} + a_2 w_{23}^{(2)} + a_3 w_{33}^{(2)} + b_3^{(2)}$
- $a_3^{(2)} = \sigma\left(z_3^{(2)}\right)$

Để nhất quán về mặt ký hiệu, gọi input layer là $a^0(= x)$ kích thước 2*1.

$$z^{(1)} = \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \\ z_3^{(1)} \end{bmatrix} = \begin{bmatrix} a_1^{(0)} w_{11}^{(1)} + a_2^{(0)} w_{21}^{(1)} + a_3^{(0)} w_{31}^{(1)} + b_1^{(1)} \\ a_1^{(0)} w_{12}^{(1)} + a_2^{(0)} w_{22}^{(1)} + a_3^{(0)} w_{32}^{(1)} + b_2^{(1)} \\ a_1^{(0)} w_{13}^{(1)} + a_2^{(0)} w_{23}^{(1)} + a_3^{(0)} w_{33}^{(1)} + b_3^{(1)} \end{bmatrix}$$

$$= (W^{(1)})^T a^{(0)} + b^{(1)}$$

Hay

$$a^{(1)} = \sigma\left(z^{(1)}\right)$$

Ví dụ 1

Cho một mạng có 2 đầu vào x_1, x_2, một lớp ẩn với 2 node và một lớp đầu ra có 1 node. Các trọng số và bias cho trước:

Input: $x_1 = 1, x_2 = 2$

Weight hidden layer:

- Node 1: $w_{11} = 0.5, w_{21} = 0.2, \text{bias}_1 = 0.1$
- Node 2: $w_{12} = -0.4, w_{22} = 0.3, \text{bias}_2 = -0.2$

Weight output layer: $w_{01} = 0.7, w_{02} = -0.5, \text{bias}_0 = 0.3$

Hàm kích hoạt Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Tính đầu ra của mạng:

Node 1:

$$z_1^{(1)} = (0.5 \times 1) + (-0.4 \times 2) + 0.1 = -0.2$$

$$a_1^{(1)} = \sigma(z_1^{(1)}) = \frac{1}{1 + e^{-(-0.2)}} \approx 0.45$$

Node 2:

$$z_2^{(1)} = (0.2 \times 1) + (0.3 \times 2) + (-0.2) = 0.6$$

$$a_2^{(1)} = \sigma(z_2^{(1)}) = \frac{1}{1 + e^{-0.6}} \approx 0.65$$

Lớp đầu ra:

$$z_0 = (0.7 \times 0.45) + (-0.5 \times 0.65) + 0.3 = 0.29$$

$$a = \sigma(z_0) = \frac{1}{1 + e^{-0.29}} \approx 0.57$$

Ví dụ 2

Cho vector đầu vào $[2, 0.1, 1]$, hãy tính giá trị đầu ra khi dùng Softmax.

Tính giá trị mũ:

$$e^2 = 7.389, \quad e^{0.1} = 1.105, \quad e^1 = 2.718$$

Tính tổng giá trị mũ:

$$\sum = 7.389 + 1.105 + 2.718 = 11.212$$

Tính giá trị đầu ra Softmax:

$$\sigma(2) = \frac{7.389}{11.212} = 0.659, \quad \sigma(0.1) = \frac{1.105}{11.212} = 0.099, \quad \sigma(1) = \frac{2.718}{11.212} = 0.242$$

2.2 Thuật toán lan truyền ngược - Backpropagation

Thuật toán Backpropagation là một phương pháp tính toán đạo hàm ngược (reverse of differentiation) được áp dụng trong mạng nơ-ron nhân tạo, giúp tính toan các đạo hàm cần thiết để tối ưu hóa các tham số trong mô hình.

Ý tưởng đầu tiên về thuật toán Backpropagation được sử dụng để tính toán đạo hàm của hàm mất mát theo các tham số trong mô hình machine learning khi áp dụng thuật toán gradient descent. Trong mô hình machine learning thì số lượng tham số có thể là rất lớn, đôi khi lên đến hàng triệu tham số. Tính toán đạo hàm của hàm mất mát theo từng tham số bằng cách sử dụng công thức tính đạo hàm bình thường có thể rất phức tạp và tốn kém về thời gian và chi phí tính toán. Thuật toán Backpropagation được thiết kế để tính toán đạo hàm của hàm mất mát theo từng tham số một cách hiệu quả hơn bằng cách sử dụng kỹ thuật lan truyền ngược (backward propagation) thông qua các lớp (layers) của mô hình. Quá trình tính toán đạo hàm này giup thuật toán gradient descent điều chỉnh các tham số mô hình sao cho giá trị của hàm mất mát giảm dần theo thời gian, giúp tối ưu hóa hiệu suất huấn luyện cho mô hình.

2.2.1 Mục tiêu của lan truyền ngược

Sự lan truyền ngược đóng vai trò quan trọng trong cách mạng lưới nơ-ron cải thiện theo thời gian. Sau đây là lý do:

Cập nhật trọng số hiệu quả : Tính toán độ dốc của hàm mất mát theo từng trọng số bằng cách sử dụng quy tắc chuỗi, giúp cập nhật trọng số một cách hiệu quả.

Khả năng mở rộng : Thuật toán truyền ngược có khả năng mở rộng tốt với các mạng có nhiều lớp và kiến trúc phức tạp, giúp học sâu trở nên khả thi.

Học tự động : Với phương pháp truyền ngược, quá trình học trở nên tự động và mô hình có thể tự điều chỉnh để tối ưu hóa hiệu suất.

2.2.2 Cách thức hoạt động của thuật toán lan truyền ngược

Tính toán độ dốc

Sự thay đổi của mỗi trọng số được tính như sau:

$$\Delta w_{ij} = \eta \times \delta_j \times O_j$$

Trong đó:

- δ_j là thuật ngữ lỗi cho mỗi đơn vị,
- η là tốc độ học tập.

Tính toán đơn vị lỗi đầu ra

Đối với O_3:

$$\delta_5 = y_5(1 - y_5)(y_{\text{target}} - y_5)$$

$$= 0.67(1 - 0.67)(-0.17) = -0.0376$$

Tính toán lỗi đơn vị ẩn

Đối với h_1:

$$\delta_3 = y_3(1 - y_3)(w_{1,3} \times \delta_5)$$
$$= 0.56(1 - 0.56)(0.3 \times -0.0376) = -0.0027$$

Đối với h_2:

$$\delta_4 = y_4(1 - y_4)(w_{2,3} \times \delta_5)$$
$$= 0.59(1 - 0.59)(0.9 \times -0.0376) = -0.0819$$

Cập nhật trọng số

Đối với trọng số từ lớp ẩn đến lớp đầu ra:

$$\Delta w_{2,3} = 1 \times (-0.0376) \times 0.59 = -0.022184$$

Trọng số mới:

$$w_{2,3}^{(\text{new})} = -0.22184 + 0.9 = 0.67816$$

Đối với trọng số từ đầu vào đến lớp ẩn:

$$\Delta w_{1,1} = 1 \times (-0.0027) \times 0.35 = 0.000945$$

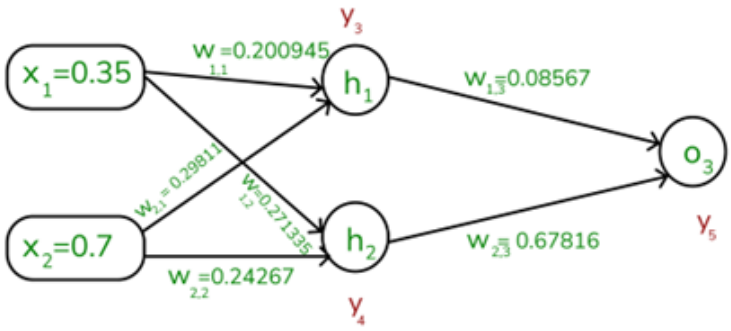
Trọng số mới:

$$w_{1,1}^{(\text{new})} = 0.000945 + 0.2 = 0.200945$$

Tương tự, các trọng số khác cũng được cập nhật:

$$w_{1,2}^{(\text{new})} = 0.271335$$
$$w_{1,3}^{(\text{new})} = 0.08567$$
$$w_{2,1}^{(\text{new})} = 0.29811$$
$$w_{2,2}^{(\text{new})} = 0.24267$$

Các trọng số được cập nhật được minh họa bên dưới.



Hình 2.2: Trọng số được cập nhật.

Sau khi cập nhật trọng số, quá trình chuyển tiếp được lặp lại, tạo ra:

$$y_3 = 0.57$$
$$y_4 = 0.56$$
$$y_5 = 0.61$$

Từ đó, $y_5 = 0.61$ vẫn chưa phải là đầu ra mục tiêu, quá trình tính toán lỗi và truyền ngược tiếp tục cho đến khi đạt được đầu ra mong muốn. Tính toán lỗi:

$$\text{Error} = y_{\text{target}} - y_5$$
$$= 0.5 - 0.61 = -0.11$$

Quá trình này sẽ tiếp tục cho đến khi mạng nơ-ron thu được kết quả đầu ra thực tế mong muốn. Quá trình truyền ngược giúp cập nhật trọng số theo chu kỳ bằng cách giảm thiểu lỗi, để mạng có thể dự đoán chính xác đầu ra.

Ví dụ:

Cho một mạng có một đầu vào $x = 0.5$, một lớp đầu ra có một node với trọng số $w = 0.8$, bias $b = 0.2$ và nhãn thực tế là $y_{\text{true}} = 1$. Hàm kích hoạt là Sigmoid, $\eta = 0.1$ và hàm mất mát là MSE (Mean Squared Error). Cập nhật trọng số w và b qua một epoch.

Các bước giải:

Đạo hàm của hàm Sigmoid theo z:

$$\sigma(z)(1 - \sigma(z))$$

Đạo hàm của hàm MSE theo y_{pred}:

$$-(y_{\text{true}} - y_{\text{pred}})$$

Lan truyền xuôi

$$z = w \cdot x + b = 0.8 \cdot 0.5 + 0.2 = 0.6$$

$$y_{\text{pred}} = \sigma(0.6) = \frac{1}{1 + e^{-0.6}} \approx 0.645$$

Tính lỗi

$$L = \frac{1}{2}(1 - 0.645)^2 \approx 0.063$$

Lan truyền ngược

$$\frac{\partial L}{\partial y_{\text{pred}}} = -(y_{\text{true}} - y_{\text{pred}}) = -(1 - 0.645) = -0.355$$

$$\frac{\partial y_{\text{pred}}}{\partial z} = \sigma(z)(1 - \sigma(z)) = 0.645(1 - 0.645) \approx 0.229$$

$$\frac{\partial z}{\partial w} = x = 0.5, \quad \frac{\partial z}{\partial b} = 1$$

$$\frac{\partial L}{\partial w} = -0.355 \times 0.229 \times 0.5 \approx -0.0406$$

$$\frac{\partial L}{\partial b} = -0.355 \times 0.229 \times 1 \approx -0.0813$$

Cập nhật trọng số

$$w' = w - \eta \cdot \frac{\partial L}{\partial w} = 0.8 - 0.1 \times (-0.0406) = 0.80406$$

$$b' = b - \eta \cdot \frac{\partial L}{\partial b} = 0.2 - 0.1 \times (-0.0813) = 0.20813$$

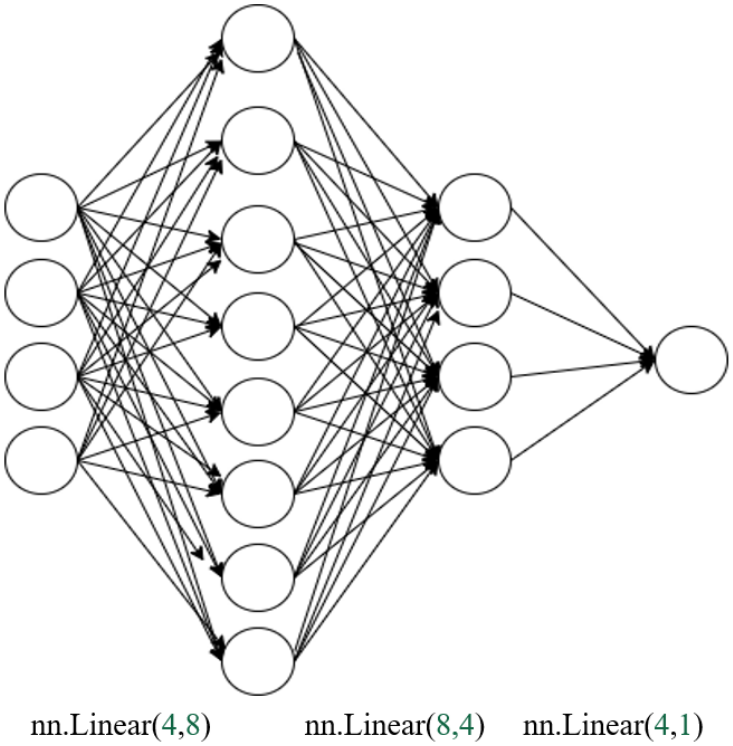
2.3 Ví dụ về sử dụng mạng truyền thẳng nhiều tầng với bài toán dự đoán giá nhà

Ví dụ về xây dựng mạng truyền thẳng với hai tầng ẩn cho bài toán dự đoán giá nhà với input đầu vào là 4 chiều như hình 2.3.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# Dinh nghia mo hinh mang truyen thang 3 tang
class HousePricePredictor(nn.Module):
    def __init__(self):
        super(HousePricePredictor, self).__init__()
        self.layer1 = nn.Linear(4, 8)
        self.layer2 = nn.Linear(8, 4)
        self.layer3 = nn.Linear(4, 1)
        self.activation = nn.ReLU() # Ham kich hoat RELU

    def forward(self, x):
        x = self.activation(self.layer1(x))
        x = self.activation(self.layer2(x))
        x = self.layer3(x)  #khong su dung ham kich hoat vi tang cuoi la lop dau ra
        return x
```



Hình 2.3: Mạng truyền thẳng 2 tầng ẩn

Chuẩn bị dữ liệu và huấn luyện mô hình:

```
# Bo du lieu huan luyen gom 4 chieu: [size, bedrooms, bathrooms, age]
data = [
    [1500, 3, 2, 15],
    [2000, 4, 3, 10],
    [1200, 2, 1, 20],
    [1800, 3, 2, 5],
    [2200, 4, 3, 8],
    [1600, 3, 2, 12],
    [2500, 5, 4, 3],
    [1300, 2, 2, 18],
    [1900, 3, 2, 7],
    [2100, 4, 3, 6]
]

# Cot gia tri price output dau ra
prices = [250, 350, 180, 300, 380, 270, 450, 200, 320, 360]

# Chuyen du lieu sang tensor
X = torch.tensor(data, dtype=torch.float32)
y = torch.tensor(prices, dtype=torch.float32).view(-1, 1)

# Chuan hoa du lieu ca X va y
X_mean, X_std = X.mean(dim=0), X.std(dim=0)
y_mean, y_std = y.mean(), y.std()
X_normalized = (X - X_mean) / X_std
y_normalized = (y - y_mean) / y_std

# Huan luyen mo hinh voi ham toi uu Adam
model = HousePricePredictor()
optimizer = optim.Adam(model.parameters(), lr=0.01)
criterion = nn.MSELoss()

# Huan luyen voi 1000 epoch
for epoch in range(1000):
    optimizer.zero_grad()
    outputs = model(X_normalized)
    loss = criterion(outputs, y_normalized)
    loss.backward()
    optimizer.step()
```

Dự đoán với dữ liệu mới:

```
def predict_price(features):
    features_normalized = (torch.tensor(features, dtype=torch.float32) - X_mean) / X_std
    with torch.no_grad():
        price_normalized = model(features_normalized.unsqueeze(0))
    return price_normalized * y_std + y_mean

# Du doan voi 10 mau du lieu
for i, house in enumerate(data):
    predicted_price = predict_price(house)
    print(f"House {i+1}:")
    print(f"Features: Size: {house[0]} sq ft, Bedrooms: {house[1]}, Bathrooms: {house[2]}")
    print(f"Age: {house[3]} years")
    print(f"Actual price: ${prices[i]},000")
    print(f"Predicted price: ${predicted_price.item()*1000:.2f}")
    print()
```

2.4 Phương pháp luận

Thành công trong việc ứng dụng các kỹ thuật học sâu đòi hỏi nhiều hơn là chỉ hiểu biết lý thuyết về thuật toán và nguyên lý hoạt động. Trên thực tế, để đạt được kết quả tốt nhất, người thực hiện cần phải có khả năng lựa chọn thuật toán phù hợp cho từng bài toán cụ thể, đồng thời biết cách giám sát và điều chỉnh quá trình thực nghiệm nhằm tối ưu hóa hiệu suất hệ thống. Trong quá trình này, nhiều quyết định quan trọng cần được đưa ra như: khi nào cần thu thập thêm dữ liệu, điều chỉnh quy mô mô hình, tinh chỉnh các đặc trưng, cải thiện thuật toán tối ưu hoặc sửa lỗi lập trình. Tất cả những kỹ năng này đều đòi hỏi thời gian tích lũy kinh nghiệm thực tế. Điều đáng chú ý là trong nhiều trường hợp, việc áp dụng đúng cách một thuật toán đơn giản thường mang lại kết quả tốt hơn đáng kể so với việc sử dụng những thuật toán phức tạp nhưng không phù hợp với bài toán.

Sau đây là các bước thiết kế một thuật toán học máy[1] [4]:

- Xác định mục tiêu học, cụ thể dùng độ đo sai số nào. Các mục tiêu và độ đo sai số nên được lựa chọn theo bài toán cần giải quyết
- Thiết kế một quy trình hoạt động đầu cuối bao gồm cả ước lượng các độ đo hiệu năng
- Lặp lại những cải thiện nhỏ như: Thu thập dữ liệu, tinh chỉnh siêu tham số, thay đổi thuật toán dựa trên kết quả thực nghiệm

2.4.1 Độ đo hiệu năng

Trong lĩnh vực học máy, việc đánh giá chính xác hiệu suất mô hình là yếu tố then chốt để đảm bảo khả năng áp dụng thực tế và tính khái quát hóa. Mỗi loại bài toán khác nhau sẽ yêu cầu những chỉ số đánh giá riêng biệt: bài toán phân loại thường sử dụng Accuracy, Precision, Recall, đặc biệt F1-Score và ROC-AUC tỏ ra hiệu quả với dữ liệu không cân bằng; bài toán hồi quy lại ưu tiên các chỉ số như MAE, MSE hay R^2 để đo lường độ chính xác của dự đoán; trong khi đó, bài toán phân cụm cần đến các chỉ số không giám sát như Silhouette Score để đánh giá chất lượng cụm. Đối với hệ thống xếp hạng, các độ đo như MAP và NDCG lại trở thành tiêu chuẩn vàng để đánh giá thứ tự kết quả. Việc lựa chọn bộ chỉ số phù hợp không chỉ phụ thuộc vào đặc thù bài toán mà còn cần xem xét tính chất dữ liệu và yêu cầu thực tế, đồng thời nên kết hợp đa dạng chỉ số để có đánh giá toàn diện.

Một nguyên tắc quan trọng khi triển khai bất kỳ dự án học máy nào là phải xác định rõ ràng ngay từ đầu các chỉ số hiệu suất cần tối ưu hóa. Bởi lẽ, nếu không có định hướng đánh giá rõ ràng, chúng ta sẽ không thể xác định được những điều chỉnh trong hệ thống thực sự mang lại cải thiện hay không, từ đó dễ dẫn đến việc cải tiến mô hình một cách mù quáng.

2.4.2 Các mô hình cơ sở mặc định

Sau khi xác định các chỉ số đánh giá và mục tiêu bài toán, cần triển khai hệ thống đầu-cuối càng sớm càng tốt. Đối với các bài toán AI-complete như nhận dạng đối tượng hoặc giọng nói, việc áp dụng ngay các mô hình học sâu thường mang lại hiệu quả vượt trội.

Lựa chọn kiến trúc mô hình phụ thuộc vào đặc điểm dữ liệu:

- Dữ liệu có giám sát dạng vector cố định → Mạng nơ-ron truyền thẳng đa tầng (MLP)

[1] https://dlbookvn.gitlab.io/about/

- Dữ liệu có cấu trúc không gian (ảnh) → Mạng tích chập (CNN)
- Dữ liệu chuỗi thời gian hoặc văn bản → Mạng truy hồi (LSTM/GRU)

Tối ưu hóa quá trình huấn luyện:

- Ưu tiên sử dụng SGD với các chiến lược điều chỉnh learning rate: giảm tuyến tính, giảm theo hàm mũ, hoặc giảm khi validation error bão hòa
- Xem xét thuật toán Adam như một lựa chọn thay thế hiệu quả
- Áp dụng các kỹ thuật chống overfitting sớm (early stopping, dropout, batch normalization) khi tập huấn luyện <10 triệu mẫu

Kế thừa giải pháp hiện có:

- Tận dụng các kiến trúc đã được chứng minh hiệu quả trên bài toán tương tự
- Chỉ áp dụng học không giám sát khi:
  + Thuộc lĩnh vực đã được chứng minh hiệu quả
  + Dữ liệu hoàn toàn không có nhãn

2.4.3 Thu thập thêm dữ liệu

Sau khi triển khai hệ thống đầu-cuối, quá trình đánh giá và cải thiện hiệu năng cần được thực hiện theo phương pháp khoa học. Khác với quan niệm phổ biến về việc tập trung vào thuật toán, các nghiên cứu thực nghiệm cho thấy việc tăng cường dữ liệu thường mang lại hiệu quả cải thiện vượt trội hơn. Tuy nhiên, quyết định này cần được xem xét kỹ lưỡng dựa trên phân tích hệ thống.

Giai đoạn đánh giá ban đầu:

1) Đối với hiệu năng trên tập huấn luyện:

- Nếu kết quả không đạt yêu cầu, cần ưu tiên:
  + Mở rộng kiến trúc mô hình (tăng số lớp/tầng)
  + Điều chỉnh siêu tham số (đặc biệt là learning rate)
- Trường hợp đã tối ưu mà không cải thiện:
  + Kiểm tra chất lượng dữ liệu (nhiễu, đặc trưng)
  + Xem xét thu thập lại dữ liệu chất lượng cao hơn

2) Đối với hiệu năng trên tập kiểm thử:

- Khi chênh lệch lớn so với tập huấn luyện:
  + Ưu tiên tăng dữ liệu huấn luyện
  + Cần đánh giá chi phí/khả thi của việc thu thập
- Trường hợp hạn chế về dữ liệu:
  + Áp dụng kỹ thuật chống overfitting (dropout, regularization)
  + Giảm độ phức tạp mô hình

Phương pháp xác định lượng dữ liệu bổ sung:

- Xây dựng đường cong học tập (learning curve) thể hiện quan hệ giữa kích thước dữ liệu và sai số
- Sử dụng phương pháp ngoại suy để dự báo lượng dữ liệu cần thiết
- Khuyến nghị thử nghiệm theo thang logarit (ví dụ: tăng gấp đôi dữ liệu mỗi lần)

Trường hợp đặc biệt:

- Đối với các ứng dụng có chi phí thu thập dữ liệu cao (ví dụ: y tế):
  + Tập trung vào kỹ thuật kiểm soát mô hình
  + Tối ưu hóa siêu tham số triệt để
- Chỉ xem xét cải tiến thuật toán khi mọi phương án khác không khả thi (thường thuộc phạm vi nghiên cứu chuyên sâu)

Quy trình này nhấn mạnh nguyên tắc ưu tiên giải pháp dữ liệu trước khi xem xét thay đổi thuật toán, đồng thời cung cấp phương pháp luận khoa học để đưa ra các quyết định tối ưu hóa hệ thống.

2.4.4 Tinh chỉnh các siêu tham số

Có hai cách tiếp cận cơ bản để lựa chọn siêu tham số:

- Lựa chọn thủ công: Cách chọn thủ công yêu cầu sự hiểu biết về vai trò của các siêu tham số và các cánh mô hình đạt được mô hình khả năng tổng quát hóa tốt
- Lựa chọn tự động: giúp giảm đáng kể việc phải hiểu thông thạo các vấn đề trên nhưng chi phí tính toán lớn

Tinh chỉnh thủ công các siêu tham số

Quá trình thiết lập siêu tham số thủ công đòi hỏi sự hiểu biết sâu sắc về mối quan hệ tương tác giữa các yếu tố: (1) giá trị siêu tham số, (2) sai số huấn luyện, (3) sai số tổng quát hóa, và (4) nguồn lực tính toán. Mục tiêu cốt lõi của việc điều chỉnh thủ công là tối ưu hóa khả năng tổng quát hóa của mô hình, cụ thể là giảm thiểu sai số trên tập kiểm thử.

Về bản chất, quá trình này nhằm điều chỉnh dung lượng hiệu dụng của mô hình sao cho phù hợp với độ phức tạp của bài toán. Dung lượng hiệu dụng chịu ảnh hưởng bởi ba yếu tố chính: (i) khả năng biểu diễn của kiến trúc mô hình, (ii) hiệu quả của thuật toán học trong việc tối ưu hàm mục tiêu, và (iii) mức độ kiểm soát của hàm chi phí đối với quá trình huấn luyện.

Đối với các siêu tham số liên quan đến quy mô mô hình, cần lưu ý rằng hiện tượng quá khớp thường xảy ra khi giá trị siêu tham số quá lớn (ví dụ: số lượng đơn vị trong tầng ẩn quá nhiều làm tăng đáng kể tham số mô hình). Ngược lại, đối với một số siêu tham số kiểm soát khác, quá khớp lại xuất hiện khi giá trị được thiết lập quá nhỏ. Do đó, việc điều chỉnh đòi hỏi phải cân nhắc kỹ lưỡng đặc tính của từng loại siêu tham số cụ thể.

Bảng 2.1: Ảnh hưởng của các siêu tham số lên dung lượng của mô hình

Siêu tham số	**Tăng dung lượng khi...**	**Lí do**	**Cảnh báo**
Số đơn vị ẩn	Tăng lên	Tăng số lượng đơn vị ẩn làm tăng khả năng biểu diễn của mô hình	Tăng số lượng đơn vị ẩn làm tăng thời gian và bộ nhớ cần thiết của mọi phép toán của mô hình.
Tốc độ học	Được tinh chỉnh một cách tối ưu	Tốc độ học quá cao hoặc quá thấp vì quá trình tối ưu thất bại	
Độ rộng của lõi tích chập	Tăng lên	Tăng độ rộng của lõi làm tăng số lượng tham số của mô hình	Sử dụng một lõi rộng hơn dẫn đến sự thu hẹp kích thước đầu ra, làm giảm dung lượng của mô hình trừ khi sử dụng kỹ thuật đệm thêm 0. Lõi rộng hơn yêu cầu nhiều bộ nhớ hơn để lưu trữ các tham số, đồng thời làm tăng thời gian chạy. Tuy nhiên đầu ra nhỏ hơn sẽ giảm bớt chi phí bộ nhớ.
Đệm thêm 0	Tăng lên	Đệm thêm 0 trước phép tích chập khiến kích thước của biểu diễn được giữ ở mức lớn	Tăng thời gian và bộ nhớ của hầu hết các phép toán.
Hệ số suy giảm trọng số (weight decay coefficient)	Giảm đi	Giảm hệ số suy giảm trọng số giải phóng các tham số của mô hình, cho phép chúng trở lên lớn hơn	
Tỉ lệ drop out	giảm đi	Khi sử dụng drop out thì số chiều ít hơn, các đơn vị có cơ hội "hợp sức"với nhau để khớp với tập huấn luyện	

Tóm lại, không phải siêu tham số nào cũng giúp ta hiểu rõ mối quan hệ giữa độ phức tạp của mô hình và sai số. Dù vậy, tốc độ học là một trong những siêu tham số quan trọng nhất. Nếu chỉ có thể điều chỉnh một siêu tham số, bạn nên ưu tiên tốc độ học. Nó quyết định khả năng học hỏi hiệu quả tối đa của mô hình khi được thiết lập ở mức phù hợp, chứ không phải quá cao hay quá thấp. Nếu tốc độ học quá lớn, thuật toán gradient descent có thể làm tăng sai số trên tập huấn luyện thay vì giảm nó. Với một hàm chi phí bậc hai lý tưởng, điều này xảy ra khi tốc độ học lớn hơn gấp đôi giá trị tối ưu. Ngược lại, tốc độ học quá nhỏ không chỉ làm chậm quá trình huấn luyện mà còn có nguy cơ khiến mô hình bị kẹt ở những vùng có sai số huấn luyện cao.

Ngoài tốc độ học, việc điều chỉnh các siêu tham số khác đòi hỏi chúng ta phải theo dõi cả hiệu suất trên tập huấn luyện và tập kiểm thử. Điều này giúp xác định xem mô hình đang gặp phải tình trạng quá khớp (overfitting) hay dưới khớp (underfitting), từ đó điều chỉnh độ phức tạp của mô hình một cách thích hợp.

Nếu sai số trên tập huấn luyện vẫn còn cao hơn mức mong muốn, giải pháp duy nhất là làm tăng khả năng biểu diễn của mô hình. Giả sử chúng ta đang sử dụng một cơ chế kiểm soát phù hợp và thuật toán tối ưu đang hoạt động tốt, thì việc thêm các lớp mới vào mạng nơ-ron hoặc tăng số lượng đơn vị trong các lớp ẩn có thể giúp ích. Tuy nhiên, việc này đồng nghĩa với việc tăng chi phí tính toán.

Khi sai số trên tập kiểm thử vượt quá ngưỡng mong muốn, chúng ta cần hiểu rằng sai số kiểm thử tốt nhất đạt được khi có sự cân bằng giữa việc giảm sai số trên tập huấn luyện và thu hẹp khoảng cách giữa sai số trên tập huấn luyện và tập kiểm thử. Thông thường, mạng nơ-ron hoạt động hiệu quả nhất khi đạt được sai số huấn luyện rất thấp (mô hình có độ phức tạp cao), và sai số kiểm thử chủ yếu phản ánh sự khác biệt giữa hiệu suất trên hai tập dữ liệu này. Mục tiêu của chúng ta lúc này là làm giảm khoảng cách đó mà không làm sai số huấn luyện tăng lên quá nhanh. Để đạt được điều này, chúng ta có thể điều chỉnh các siêu tham số của cơ chế kiểm soát nhằm giảm độ phức tạp hiệu quả của mô hình, ví dụ như áp dụng dropout hoặc tăng cường hệ số suy giảm trọng số.

Hầu hết các siêu tham số có thể được thiết lập bằng cách xem xét liệu chúng sẽ làm tăng hay giảm dung lượng của mô hình như trong bảng 2.1.

Khi tinh chỉnh siêu tham số theo cách thủ công, điều quan trọng nhất là tối ưu hóa kết quả trên tập kiểm tra. Các kỹ thuật điều chuẩn chỉ là một trong số các công cụ để đạt được mục tiêu này. Nếu kết quả trên tập kiểm tra vẫn tốt, việc thu thập thêm dữ liệu huấn luyện có thể giúp cải thiện khả năng khái quát hóa của mô hình. Trong một số trường hợp, để đảm bảo thành công, người ta có thể tăng dần độ lớn của mô hình và tập dữ liệu huấn luyện cho đến khi đạt được kết quả mong muốn. Tuy nhiên, cách tiếp cận này có thể tốn kém về mặt tính toán, đặc biệt là trong quá trình huấn luyện và suy luận, và do đó cần được cân nhắc kỹ lưỡng dựa trên các nguồn lực sẵn có.

Thuật toán tinh chỉnh siêu tham số tự động

Tinh chỉnh thủ công các siêu tham số có thể mang lại kết quả tốt nếu người thực hiện có một điểm khởi đầu tốt, chẳng hạn như khi các siêu tham số đã được xác định từ các nghiên cứu trên các ứng dụng và kiến trúc tương tự, hoặc khi người dùng đã dành nhiều thời gian để tìm ra các giá trị siêu tham số tối ưu cho mạng nơ-ron được áp dụng trong một tác vụ tương tự. Tuy nhiên, khi bắt đầu với một bài toán mới mà không có điểm khởi đầu tốt, các thuật toán tự động có thể giúp chúng ta tìm thấy các siêu tham số hữu ích.Về cơ bản, chúng ta thực hiện một thuật toán tối ưu hóa siêu tham số, bao quanh một thuật toán học, để chọn các siêu tham số cho thuật toán đó. Tuy nhiên, thuật toán tối ưu hóa siêu tham số cũng có các siêu tham số riêng của nó. Bộ siêu tham số thứ hai này thường dễ chọn hơn; chúng ta có thể đạt được hiệu suất chấp nhận được trên nhiều tác vụ khác nhau bằng cách sử dụng cùng một bộ siêu tham số thứ hai cho tất cả các tác vụ.

Dưới đây là một số thuật toán tối ưu siêu tham số tự động:

- Tìm kiếm theo lưới: Khi cần lựa chọn từ 3 siêu tham số trở xuống, thì phương pháp tìm kiếm theo lưới (grid search) thường được sử dụng. Với mỗi siêu tham số, người dùng cần chọn một tập nhỏ, hữu hạn các giá trị để thử nghiệm. Thuật toán tìm kiếm theo lưới sẽ huấn luyện một mô hình với mọi tổ hợp cấu hình của các tham số lấy từ tích Descartes của các tập giá trị của mỗi siêu tham số. Phép thử nào cho kết quả tốt nhất trên tập kiểm thử sẽ được chọn.

  Danh sách các giá trị thử nghiệm nên được chọn như sau: Trong trường hợp siêu tham số có dạng số (và được sắp xếp), phần tử nhỏ nhất và lớn nhất của mỗi danh sách cần được chọn một cách cẩn trọng dựa theo kinh nghiệm trước đó ở các thử nghiệm tương tự, để đảm bảo các giá trị tối ưu có nhiều khả năng nằm trong miền được chọn. Khi tìm kiếm theo lưới, chúng ta thường chọn giá trị trên thang logarit. Ví dụ, chọn tốc độ học trong tập 0.1, 0.001, 0.0001, 0.00001, hay chọn số chiều trong tầng ẩn trong tập 50, 100, 200, 500, 1000, 2000.

  Tìm kiếm theo lưới thường đạt kết quả tốt nhất khi được thực hiện nhiều lần. Ví dụ, giả sử ta cần tìm kiếm siêu tham số α với miền giá trị -1, 0, 1. Nếu giá trị tốt nhất tìm được là 1, thì nhiều khả

năng chúng ta đã đánh giá thấp miền giá trị chứa α tốt nhất, vì vậy ta nên dịch lưới lên để tìm giá trị α trong tập 1, 2, 3. Nếu giá trị tốt nhất tìm được là 0, ta có thể hi vọng tìm được α tìm được tốt hơn khi phóng to khoảng tìm kiếm bằng cách thực hiện tìm kiếm trong tập -0.1, 0, 0.1.

Một vấn đề của tìm kiếm lưới đó là chi phí tính toán của nó tăng theo hàm mũ của số lượng siêu tham số. Nếu có m tham số, mỗi tham số có nhiều nhất n giá trị, khi đó số phép thử cho quá trình huấn luyện và đánh giá có thể lên tới $O(n^m)$. Mặc dù các phép thử có thể song song nhưng chi phí hàm mũ thì ngay cả việc song song cũng không cung cấp được một tìm kiếm hợp lý.

- Tìm kiếm ngẫu nhiên: Một tìm kiếm thay thế cho tìm kiếm lưới dễ tương tự nhưng hiệu quả và hội tụ nhanh hơn tới các giá trị tốt của siêu tham số.

  Tìm kiếm ngẫu nhiên hoạt động như sau: Đầu tiên, chúng ta xác định phân phối biên cho mỗi siêu tham số. Không như tìm kiếm lưới, ở phương pháp này chúng ta không nên thực hiện rời rạc hóa giá trị của các siêu tham số để có thể khám phá tập giá trị lớn hơn và tránh phát sinh chi phí tính toán. Tìm kiếm ngẫu nhiên có thể hiệu quả hơn theo cấp số nhân so với tìm kiếm theo lưới nếu có một vài siêu tham số không ảnh hưởng nhiều đến hiệu năng. Các nghiên cứu cũng đã chỉ ra rằng tìm kiếm ngẫu nhiên giúp giảm sai số trên tập kiểm thử nhanh hơn nhiều so với tìm kiếm theo lưới, xét theo số phép thử được thực thi bởi mỗi thuật toán.

  Tương tự với kiểm thử theo lưới, chúng ta cũng lặp lại quá trình tìm kiếm ngẫu nhiên để tinh chỉnh phạm vi tìm kiếm dựa theo kết quả của lần thực thi trước.

  Giải thích lý do cho tìm kiếm ngẫu nhiên thường tìm được cấu hình tốt và nhanh hơn tìm kiếm theo lưới là do không thực hiện phép thử vô ích có thể xuất hiện trong tìm kiếm theo lưới. Hai giá trị khác nhau của siêu tham số (khi biết giá trị của siêu tham số còn lại) cho kết quả tương đương. Đối với tìm kiếm theo lưới, các siêu tham số còn lại có thể nhận giá trị giống nhau trong hai phép thử này, trong khi với tìm kiếm ngẫu nhiên, các siêu tham số còn lại thường nhận những giá trị khác nhau. Do vậy, nếu việc thay đổi hai giá trị này không tạo ra nhiều khác biệt trên sai số kiểm thử, thì tìm kiếm theo lưới sẽ lặp lại hai phép thử giống nhau một cách không cần thiết, còn tìm kiếm ngẫu nhiên sẽ tạo ra hai phép thử độc lập cho các siêu tham số còn lại.

- Tối ưu siêu tham số bằng mô hình: Quá trình tìm kiếm siêu tham số tối ưu có thể quy về một bài toán tối ưu, với biết quyết định là các siêu tham số. Chi phí cần được tối ưu chính là sai số kiểm thử của quá trình huấn luyện sử dụng các siêu tham số này. Với cách làm này có thể tính đạo hàm theo các siêu tham số của một độ đo sai số khả vi trên tập kiểm thử. Tuy nhiên trong hầu hết các tình huống thực tế, việc tính đạo hàm không khả thi bởi vì nó yêu cầu chi phí tín htoasn và chi phí bộ nhớ cao. Để khắc phục nhược điểm không tính được đạo hàm, ta có thể xây dựng một mô hình của sai số kiểm thử, sau đó đề xuất các dự đoán mới cho siêu tham số bằng cách thực hiện tối ưu trong mô hình này. Phần lớn các thuật toán tìm siêu tham số dựa trên mô hình đều sử dụng mô hình hồi quy bayes để ước tính giá trị kì vọng của sai số kiểm thử cho mỗi siêu tham số và mức độ bất định xung quanh kì vọng này. Do vậy, quá trình tối ưu phải đánh đổi giữa việc khám phá và đề xuất siêu tham số có độ bất định cao, có khả năng dẫn đến sự cải thiện lớn nhưng cũng có thể cho kết quả không tốt. Do vậy, phương pháp tối ưu siêu tham số bằng mô hình không phải là phương pháp tiếp cận mặc định trong mọi trường hợp. Tối ưu tham số kiểu Bayes đôi khi có thể so sánh ngang với con người, đôi khi tốt hơn, nhưng có khi bị thất bại thảm hại trong một số bài toán.

  Nhược điểm thường thấy của hầu hết các thuật toán tối ưu siêu tham số phức tạp hơn so với tìm kiếm ngẫu nhiên bởi vì chúng ta cần hoàn thành một lần huấn luyện thử nghiệm thì mới có thể trích xuất thông tin từ đó. Điều này làm cho thuật toán tìm kiếm có hiệu suất kém hơn với với tìm kiếm thủ công do con người, xét về khía cạnh số lượng thông tin có thể trích xuất từ giai đoạn đầu của thử nghiệm, bởi vì chúng ta có thể nhận ra một tập siêu tham số hoàn toàn không hiệu quả.

Tóm lại, tinh chỉnh siêu tham số là một bước quan trọng trong quá trình phát triển và tối ưu hóa các mô hình học sâu. Siêu tham số (hyperparameter) là các tham số không được học trong quá trình huấn luyện mà được thiết lập trước khi quá trình huấn luyện bắt đầu. Chúng quyết định hành vi và khả năng học của mô hình. Một số ví dụ về siêu tham số bao gồm tốc độ học (learning rate), kích thước batch (batch size), số lớp mạng (number of layers), và thuật toán tối ưu (optimizer).

Mục tiêu của việc tinh chỉnh siêu tham số là tìm ra tập hợp các giá trị tối ưu cho siêu tham số nhằm tối đa hóa hiệu suất của mô hình trên một nhiệm vụ cụ thể, thường được đo lường qua tập dữ liệu kiểm định (validation set). Việc chọn sai siêu tham số có thể dẫn đến hiện tượng underfitting hoặc overfitting, làm giảm khả năng tổng quát hóa của mô hình.

Siêu tham số trong học sâu có thể được chia thành các nhóm chính sau:

- Siêu tham số liên quan đến mô hình:

  - Số lớp trong mạng nơ-ron.
  - Số lượng neuron mỗi lớp.
  - Hàm kích hoạt (ví dụ: ReLU, sigmoid).
- Siêu tham số liên quan đến huấn luyện
  - Tốc độ học (learning rate).
  - Kích thước batch (batch size).
  - Số lượng epoch.
  - Thuật toán tối ưu (ví dụ: Adam, SGD).
- Siêu tham số liên quan đến regularization
  - Tỷ lệ dropout.
  - Weight decay.
  - Hệ số regularization L1/L2.
- Siêu tham số liên quan đến dữ liệu
  - Kỹ thuật tăng cường dữ liệu (data augmentation).
  - Các bước tiền xử lý (ví dụ: chuẩn hóa, scaling).

Tinh chỉnh siêu tham số là một quy trình lặp lại và tiêu tốn nhiều tài nguyên nhưng đóng vai trò quan trọng trong việc cải thiện hiệu suất của các mô hình học sâu. Với sự phát triển của các công cụ tối ưu hóa tự động, các nhà phát triển có thể dễ dàng tìm kiếm cấu hình siêu tham số tối ưu. Kết hợp các công cụ này với kiến thức chuyên môn và các kỹ thuật xác thực đảm bảo tạo ra các mô hình mạnh mẽ và có khả năng tổng quát tốt. Bảng 2.2 mô tả độ nhậy (gần đúng) của các siêu tham số ảnh hưởng tới hiệu năng của mạng nơ ron.

Bảng 2.2: Độ nhậy của các siêu tham số trong học sâu

Siêu tham số	**Độ nhậy (gần đúng) của siêu tham số**
Tốc độ học	cao
Optimizer choice	thấp
Các tham số khác (e.g., Adam beta1)	thấp
Batch size	thấp
Khởi tạo trọng số	trung bình
Hàm mục tiêu	cao
Độ sâu của mô hình	trung bình
Số chiều của tầng ẩn	cao
Layer params (ví dụ: kernel size)	trung bình
Trọng số hiệu chỉnh	trung bình
Nonlinearity	thấp

2.4.5 Chiến lược gỡ lỗi

Khi một hệ thống hoạt động kém có hai nhận định:

- Do bản chất của thuật toán.
- Lỗi trong lập trình.

Hầu hết các chiến lược gỡ lỗi trong mạng nơ ron được thiết kế để giải quyết hai khó khăn trên, bằng cách thiết kế một kịch bản đơn giản để dự đoán hành vi thực sự của thuật toán hoặc thiết kế phép thử để kiểm tra từng thành phần của mạng nơ ron một cách cô lập. Một số phép thử được thiết kế như sau:

- Quan sát cách mô hình hoạt động. Ví dụ khi huấn luyện một mô hình phát hiện đối tượng trong ảnh, hãy quan sát một số ảnh cùng kết quả phát hiện của mô hình. Khi huấn luyện mô hình sinh tiếng nói, hãy lắng nghe một số mẫu tiếng nói mà mô hình sinh ra. Việc này rất dễ bị bỏ qua trong thực tiễn bởi vì chúng ta chỉ nhìn vào độ đo hiệu năng định lượng của mô hình. Trực tiếp quan sát cách mô hình học máy thực hiện tác vụ sẽ giúp chúng ta xác định xem hiệu năng định lượng mà nó đạt được có hợp lý hay không. Lỗi đánh giá hiệu năng có thể là một trong những lỗi gây hậu quả nghiêm trọng vì chúng có thể khiến bạn tin rằng hệ thống đang hoạt động tốt nhưng thực tế lại không phải như vậy.

- Quan sát các mẫu sai tồi tệ nhất. Ví dụ, các bộ phận phân loại sử dụng tầng softmax đầu ra chỉ để xác định xác suất cho mỗi loại. Do đó, xác suất được gán cho loại có khả năng cao nhất. Thông thường, quá trình huấn luyện bằng nguyên tắc hợp lí cực đại khiến cho những giá trị này vượt quá độ tự tin thực sự của mô hình, nhưng chúng vẫn hữu ích bởi vì những mẫu khó được gán nhãn chính xác hơn thường nhận xác suất nhỏ hơn từ mô hình. Vì vậy, bằng cách quan sát các mẫu khó học nhất, ta thường phát hiện ra các vấn đề trong cách tiền xử lý dữ liệu hoặc gán nhãn.

- Suy đoán về phần mềm thông qua sai số huấn luyện và kiểm thử. Để xác định phần mềm có được code đúng hay không thường rất khó. Sai số huấn luyện và sai số kiểm thử có thể giúp chúng ta xác định manh mối về lỗi. Nếu sai số huấn luyện thấp nhưng sai số kiểm thử cao thì có vẻ quá trình huấn luyện hoạt động chính xác và mô hình đang bị quá khớp. Một khả năng khác là sai số kiểm thử bị đánh giá một cách không chính xác do bị lỗi khi lưu mô hình sau khi huấn luyện và tải mô hình lên để đánh giá trên tập kiểm thử. Một lỗi khác có thể xảy ra là phân phối trên tập dữ liệu kiểm thử khác với phân phối trên tập huấn luyện. Nếu cả sai số huấn luyện và sai số kiểm thử đều cao thì lại khó để xác định xem nguyên nhân gây ra là lỗi lập trình hay do mô hình bị dưới khớp. Với trường hợp này cần thực hiện kịch bản kiểm thử được mô tả tiếp theo đây.

- Khớp mô hình với một tập dữ liệu rất nhỏ. Nếu sai số trên tập huấn luyện cao, hãy xác định xem nguyên nhân là do dưới khớp hay là do lập trình. Thông thường, ngay cả các mô hình nhỏ cũng có thể đảm bảo khả năng khớp với tập dữ liệu nhỏ. Ví dụ, một tập dữ liệu phân loại với chỉ một mẫu có thể được khớp bằng cách thiết lập chính xác các hệ số tự do của tầng đầu ra. Nếu bạn không thể huấn luyện một bộ phân loại để gán nhãn chính xác cho một mẫu đơn giản, thì không thể huấn luyện một bộ mã hóa để tái hiện thành công một mẫu duy nhất với độ tin cậy cao, hay không thể huấn luyện một mô hình sinh mẫu nhất quán sinh ra các mẫu tương đồng với một mẫu duy nhất, nghĩa là có lỗi phần mềm nào đó ngăn cản quá trình tối ưu thành công trên tập huấn luyện. Phép thử này có thể được mở rộng cho tập dữ liệu nhỏ gồm vài mẫu thay vì chỉ dùng 1 mẫu.

- So sánh đạo hàm tính bởi thuật toán lan truyền ngược với đạo hàm số học. Trường hợp này xảy ra khi bạn tự lập trình các bước tính đạo hàm hoặc bạn đang muốn thêm một phép toán mới vào thư viện tính đạo hàm thì nguyên nhân phổ biến là do lập trình không chính xác.

- Giám sát biểu đồ tần suất của tín hiệu kích hoạt và đạo hàm. Một thao tác cung cấp nhiều thông tin hữu ích là hiển thị thống kê đạo hàm và tín hiệu kích hoạt trong mạng nơ ron, được thu thập qua một lượng lớn vòng lặp huấn luyện. Giá trị tiền kích hoạt (là giá trị trước khi đi qua hàm kích hoạt) của các đơn vị ẩn có thể cho biết liệu các đơn vị ẩn có bị bão hòa hay không, hoặc tần suất bão hòa của chúng. Ví dụ các nơ ron sử dụng hàm kích hoạt RELU, chúng có thường bị tắt đi hay không. Có đơn vị nào mà luôn bị tắt không. Còn hàm tanh, giá trị tuyệt đối trung bình của các giá trị tiền kích hoạt cho biết mức độ bão hòa của đơn vị này. Trong một mạng nơ ron nhiều tầng, nếu các giá trị gradient bị bùng nổ hoặc tiêu biến quá nhanh thì quá trình tối ưu có thể bị cản trở. Ngoài ra việc so sánh độ lớn của gradient theo từng tham số với độ lớn của tham số này cũng rất hữu ích. Bằng thực nghiệm, các nhà nghiên cứu đã chỉ ra rằng, độ lớn bước cập nhật tham số tính trên 1 lô nhỏ nên có giá trị bằng khoảng 1% độ lớn của tham số đó. Nếu độ lớn đó mà là 50% hoặc 0.002% khiến quá trình cập nhật tham số trở nên rất chậm. Đôi khi, có một số nhóm các tham số được cập nhật với tốc độ phù hợp trong khi các tham số khác lại bị mắc kẹt. Đối với dữ liệu thưa như NLP, một vài tham số rất hiếm khi được cập nhật. Do vậy, chúng ta cần lưu ý khi giám sát quá trình cập nhật tham số.

2.5 Bài tập

Bài 1

Cho một mạng có 2 đầu vào x_1, x_2, một lớp ẩn với 2 node và một lớp đầu ra có 1 node. Các trọng số và bias cho trước:

Input: $x_1 = 2$, $x_2 = 3$
Weight hidden layer:
- Node 1: $w_{11} = 0.6$, $w_{21} = -0.3$, $\text{bias}_1 = 0.2$
- Node 2: $w_{12} = 0.1$, $w_{22} = 0.5$, $\text{bias}_2 = -0.1$
Weight output layer: $w_{01} = 0.8$, $w_{02} = -0.6$, $\text{bias}_0 = 0.4$
Hàm kích hoạt: Sigmoid. Hãy tính giá trị đầu ra của mạng.

Bài 2

Cho một mạng có 2 đầu vào x_1, x_2, một lớp ẩn với 2 node và một lớp đầu ra có 1 node. Các trọng số và bias cho trước:
Input: $x_1 = 0.5$, $x_2 = 1.5$
Weight hidden layer:
- Node 1: $w_{11} = -0.2$, $w_{21} = 0.4$, $\text{bias}_1 = 0.1$
- Node 2: $w_{12} = 0.3$, $w_{22} = -0.1$, $\text{bias}_2 = -0.2$
Weight output layer: $w_{01} = 0.5$, $w_{02} = 0.7$, $\text{bias}_0 = -0.3$
Hàm kích hoạt: ReLU. Hãy tính giá trị đầu ra của mạng.

Bài 3

Cho một mạng có 2 đầu vào x_1, x_2, một lớp ẩn với 2 node và một lớp đầu ra có 1 node. Các trọng số và bias cho trước:
Input: $x_1 = 1.5$, $x_2 = 2.5$
Weight hidden layer:
- Node 1: $w_{11} = 0.4$, $w_{21} = -0.2$, $\text{bias}_1 = 0.05$
- Node 2: $w_{12} = -0.3$, $w_{22} = 0.6$, $\text{bias}_2 = 0.1$
Weight output layer: $w_{01} = -0.7$, $w_{02} = 0.9$, $\text{bias}_0 = 0.2$
Hàm kích hoạt: Sigmoid. Hãy tính giá trị đầu ra của mạng.

Bài 4

Cho vector đầu vào $[4, 2, 0.5]$, hãy tính giá trị đầu ra khi sử dụng hàm Softmax.

Bài 5

Cho vector đầu vào $[3, 1.5, 0.5, 2]$, hãy tính giá trị đầu ra khi sử dụng hàm Softmax.

Bài 6

Tính đạo hàm cho hàm Sigmoid, ReLU, Tanh.

Bài 7

Tính đạo hàm cho hàm MSE, RMSE, MAE.

Bài 8

Cho một mạng có 1 đầu vào $x = 0.7$, một lớp đầu ra có 1 node với $w = 0.6$, $\text{bias} = 0.1$ và nhãn thực tế là $y_{\text{true}} = 0$. Hàm kích hoạt là ReLU, $\eta = 0.1$ và hàm mất mát là MAE (Mean Absolute Error). Cập nhật trọng số w và b qua 1 epoch.

Bài 9

Viết chương trình Python xây dựng một mạng nơ-ron có một lớp ẩn để huấn luyện cho bài toán XOR.

Bài 10

Viết chương trình Python huấn luyện một mạng nơ-ron để dự đoán giá nhà dựa trên diện tích.